

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 3 | AULA 01 -

FUNDAMENTOS DE SEGURANÇA EM APLICAÇÕES

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
O QUE VOCÊ VIU NESTA AULA? 30
REFERÊNCIAS 31
PALAVRAS-CHAVE 32

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Começaremos nossa jornada pela segurança em aplicações Java, na qual construiremos um projeto com Spring Security para uma autenticação simples por usuário e senha, evoluindo a seguir para o uso do Basic Authentication em memória. Ao final, criaremos um serviço com autenticação via formulário e controle de acesso por perfis de usuário (Role-Based Access Control ou RBAC) com banco de dados.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Os projetos que criaremos aqui serão importantes para pavimentar o conhecimento sobre a segurança das aplicações e garantir a prática de como fazemos isso em projetos reais. O Spring Security é um dos frameworks mais amplamente utilizados na comunidade Java para a proteção de aplicações web e APIs. Essa ferramenta poderosa oferece recursos que facilitam a implementação de autenticação e autorização, além de fornecer um ambiente seguro e escalável para aplicações empresariais.

Falaremos sobre os principais desafios na segurança das aplicações Java e como ela é vital para o sucesso do projeto de software que estamos implementando. Além disso, demostraremos a implementação de um sistema de autenticação básica por usuário e senha e criaremos um serviço de autenticação via formulário e controle de acesso RBAC.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

Para iniciar nossa jornada nos **Fundamentos de segurança em aplicações Java com Spring Security**, é essencial entender a importância e os conceitos básicos da segurança em sistemas modernos.

A segurança da informação é um dos pilares fundamentais na proteção de dados e na garantia de integridade e confiabilidade dos sistemas que construímos. Com o aumento das ameaças digitais, como ataques de malware, vazamentos de dados e invasões, a necessidade de arquiteturas seguras tornou-se vital, tanto para a proteção dos usuários quanto para a conformidade com regulamentos, como a **LGPD**.

Para criar os projetos dessa aula, será importante preparar o ambiente de desenvolvimento. Para fazer isso, siga os passos abaixo:

1. Baixar e instalar a versão Java jdk 21.

   Na release **21.0.2**, clique no link Binaries.
   Selecione a opção:
   OpenJDK21U-debugimage_x64_windows_hotspot_21.0.2_13.zip

2. Adicionar a variável de ambiente **JAVA_HOME**:

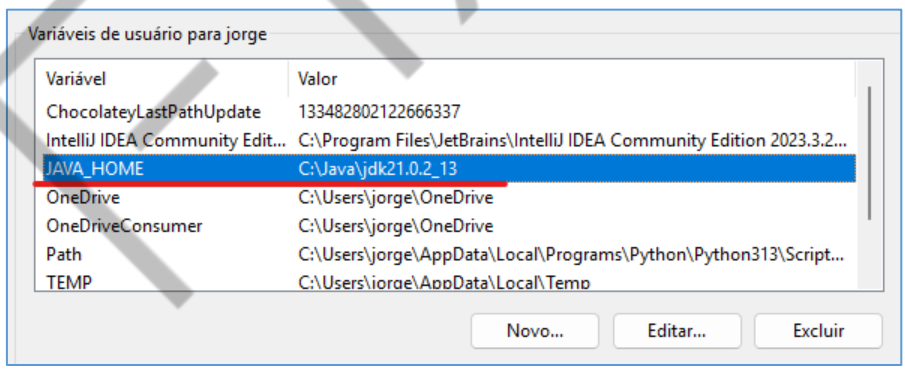


Figura 1 – Criação da Variável de Ambiente JAVA_HOME
Fonte: elaborado pelo autor (2025)

3. Baixar e instalar o IntelliJ IDEA Community.
4. Baixar o Postman.
5. Instalar o Fiddler. Ele é usado para executar capturas de requisições HTTP(S).

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

6. Vamos acessar o Spring Initializr, uma aplicação do Spring Framework que nos ajudará a criar o projeto.

Agora, vamos observar os pontos mais importantes que vimos nas videoaulas?

Para adicionar novas dependências em seu projeto, acesse o menu **Code → Generate Alt+Insert** e selecione a opção **Managed dependency**. Digite o nome da biblioteca a ser pesquisada, nesse caso, **Spring Starter Security**, e aparecerá a lupa de busca. Ao final, clique no botão **OK.**

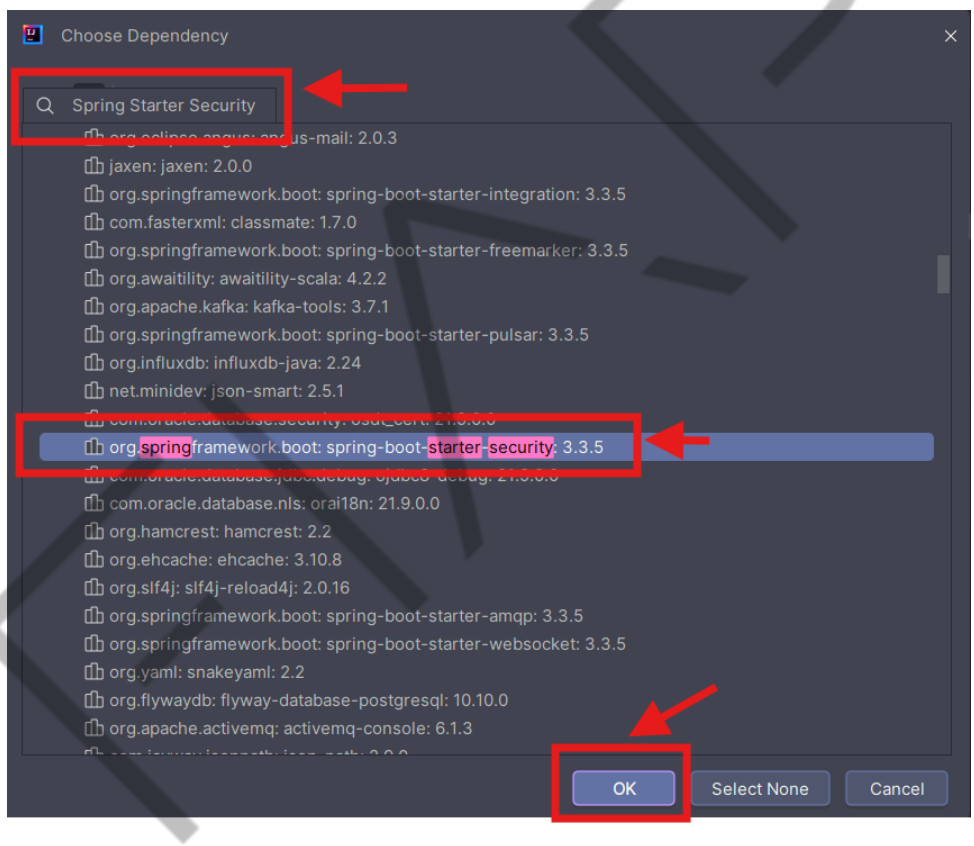


Figura 2 – Adicionando dependências no POM.XML
Fonte: elaborado pelo autor (2025)

Com o **Spring Security 6.1**, um outro ponto importante de mudança é a criação da classe de configuração abaixo:

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
    http.authorizeHttpRequests(
            customizer -> {
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
                customizer.requestMatchers("/public").permitAll();
                customizer.requestMatchers("/logout").permitAll();
                customizer.anyRequest().authenticated();
            });
    return http.build();
}
```

Código-fonte 1 – Criação de classe
Fonte: elaborado pelo autor (2025)

Temos, então, que adicionar a anotação **@Configuration** e ela terá outra anotação especial, que é a **@EnableWebSecurity**, sendo reconhecida como a classe que habilitará a segurança para o serviço web.

Antes, era preciso estender (**extends**) a classe **WebSecurityConfigurerAdapter**, mas essa classe foi removida na nova versão do Spring Security. O Spring tem feito um trabalho de retirar essas extensões e adotar as anotações, permitindo um desacoplamento maior do framework.

Outro ponto importante é que a segurança é feita através de filtros (**filters**). Assim como um filtro de café, por onde entra a água, se mistura com o pó e, do outro lado, sai o café como produto final, nos **filters** entram requisições web, interceptando uma requisição e checando se ela é autenticada ou não. Dependendo do resultado, não mandará o controlador executar a lógica de negócio na service.

Vamos então adicionar um **@Bean** chamado **SecurityFilterChain**. É uma chain porque são vários filtros de segurança, uma cadeia para customizar a autenticação e autorização. Ele recebe como parâmetro um objeto **HttpSecurity** para configurar a segurança. Vou usar a sintaxe de **Lambda Function** para expor a configuração de autorização para cada rota. Ao final, faço um **build** para retornar o resultado. A primeira coisa que devemos fazer é dizer que qualquer requisição deve ser feita por alguém autenticado. O Spring Security trabalha com essa filosofia.

Seguindo com o desenvolvimento, outro ponto importante é a criação da classe **MyUserDetailService.**

```
package com.fiap.aula1.service;

import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```java
import org.springframework.stereotype.Service;

import java.util.ArrayList;

@Service
public class MyUserDetailService implements UserDetailsService {

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        return new User("usuario", "senha", new ArrayList<>());
    }
}
```

Código-fonte 2 – MyUserDetailService
Fonte: elaborado pelo autor (2025)

Esse método retorna, somente para exemplo, um usuário **hard-coded.** Em um projeto real, o usuário deve vir de outra fonte, como uma base de dados. O Spring vai comparar as credenciais (usuario/senha) recebido pela aplicação com essa credencial (usuario/senha) **hard-coded** (usuario = **usuario** e senha = **senha**).

Queremos que o endpoint **/private** seja acessado apenas por usuários válidos. Por isso, vamos criar o serviço de autenticação de nome **MyUserDetailService**, que implementará a interface **UserDetailsService**, uma interface que faz parte da biblioteca do **Spring Security**. O objetivo dessa classe é usar o método sobrescrito **loadUserByUsername** para carregar dados de um usuário, que serão usados posteriormente pelo **Spring Security** para autenticar o usuário tentando acesso. Apenas para exemplo, os dados estão fixos, ignorando a lista de autorizações no último parâmetro **new ArrayList<>()**.

Bem, agora que já temos um serviço de autenticação, precisamos informar ao Spring Security que usaremos uma validação por usuário e senha e que o Spring Security deve acionar o nosso serviço criado no passo anterior. Para isso, vamos **modificar a implementação da classe de configuração SecurityConfig.**

```java
private final MyUserDetailService myUserDetailService;

public SecurityConfig(MyUserDetailService myUserDetailService) {
    this.myUserDetailService = myUserDetailService;
}
```

Código-fonte 3 – SecurityConfig
Fonte: elaborado pelo autor (2025)

private final MyUserDetailService myUserDetailService;

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Essa linha declara uma variável myUserDetailService do tipo MyUserDetailService, que será usada para fornecer os detalhes do usuário durante a autenticação. O uso do modificador final indica que essa variável é imutável após sua atribuição.

public SecurityConfig(MyUserDetailService myUserDetailService)

Este é o construtor da classe **SecurityConfig**. Ele injeta a dependência **MyUserDetailService**, que é um serviço que implementa a interface **UserDetailsService** e carrega os detalhes do usuário (como nome e senha). Esse serviço é essencial para o Spring Security gerenciar a autenticação baseada em usuários.

Agora, vamos implementar o método que permite configurar o **AuthenticationManager** e o disponibilizar como um **bean** gerenciado pelo Spring. Isso é especialmente útil para casos em que outras classes na aplicação (como filtros de autenticação personalizados ou controladores) precisam acessar diretamente o **AuthenticationManager** para autenticar requisições ou validar credenciais.

```
@Bean
public AuthenticationManager
authenticationManager(AuthenticationConfiguration
authenticationConfiguration) throws Exception {
    return authenticationConfiguration.getAuthenticationManager();
}
```

Código-fonte 4 – AuthenticationManager
Fonte: elaborado pelo autor (2025)

@Bean

Essa anotação indica que o método authenticationManager retorna um bean (componente) gerenciado pelo contêiner Spring. Isso significa que o AuthenticationManager retornado por esse método estará disponível para injeção em outras partes da aplicação onde a autenticação é necessária.

public AuthenticationManager authenticationManager(...)

Esse método está definindo um bean do tipo AuthenticationManager, que é o componente central do Spring Security responsável pela autenticação de usuários.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

(AuthenticationConfiguration authenticationConfiguration)

Esse é um parâmetro do tipo AuthenticationConfiguration, que é injetado automaticamente pelo Spring. O AuthenticationConfiguration fornece as configurações de autenticação da aplicação. Ele contém informações sobre como os detalhes da autenticação devem ser configurados e quais AuthenticationManager estão disponíveis para autenticar usuários.

throws Exception

A declaração throws Exception indica que esse método pode lançar uma exceção do tipo Exception. Isso ocorre porque o método getAuthenticationManager() pode lançar uma exceção, caso haja algum problema ao obter o AuthenticationManager.

return authenticationConfiguration.getAuthenticationManager();

Esse comando retorna uma instância do AuthenticationManager que é obtida a partir do objeto authenticationConfiguration. O método getAuthenticationManager() faz o trabalho de construir e configurar o AuthenticationManager com base na configuração de autenticação definida em outras partes do sistema (como configurações de provedores de autenticação, detalhes do usuário e regras de autorização).

Agora, precisamos de métodos que definem uma implementação personalizada de um PasswordEncoder, que armazena senhas em texto puro, sem qualquer tipo de criptografia.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new PlainTextPasswordEncoder();
}

public static class PlainTextPasswordEncoder implements PasswordEncoder {
    @Override
    public String encode(CharSequence rawPassword) {
        return rawPassword.toString(); // Retorna a senha como está, sem criptografia
    }

    @Override
    public boolean matches(CharSequence rawPassword, String encodedPassword) {
        return rawPassword.toString().equals(encodedPassword); // Compara diretamente a senha
    }
}
```

Código-fonte 5 – passwordEnconder

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Fonte: elaborado pelo autor (2025)

passwordEncoder

Este método está anotado com @Bean, o que indica ao Spring que ele deve ser gerenciado como um bean no contexto da aplicação. Ele retorna uma instância da **inner class PlainTextPasswordEncoder**, que implementa a interface **PasswordEncoder** do Spring Security. **PasswordEncoder** é uma interface que define métodos para codificação e verificação de senhas, geralmente usada para aplicar hashing ou criptografia nas senhas antes de armazená-las.

PlainTextPasswordEncoder

Esta é uma classe estática interna que implementa a interface **PasswordEncoder**. Como o nome sugere, ela **não realiza nenhuma criptografia ou hashing**, armazenando as senhas em texto puro.

Método encode

Recebe a senha bruta (rawPassword) como um CharSequence e a converte para uma String usando toString(). Em seguida, retorna o valor da senha exatamente como foi recebida, sem aplicar criptografia ou hashing. O propósito padrão do método encode é transformar a senha em um formato seguro para armazenamento. Aqui, no entanto, ele não faz nenhuma transformação, o que torna essa implementação insegura em termos de armazenamento de senhas.

Método matches

Recebe dois parâmetros: rawPassword, que é a senha fornecida pelo usuário, e encodedPassword, que é a senha armazenada (não criptografada, neste caso). O método compara diretamente os valores da senha fornecida com a senha armazenada usando equals(). Isso significa que as senhas precisam coincidir exatamente em termos de texto. Como não há hashing ou criptografia, o método matches apenas verifica se as duas strings são idênticas.

Importante: essa abordagem de armazenar senhas em texto puro é altamente insegura e geralmente não é recomendada para sistemas de produção, pois senhas em texto claro são vulneráveis a ataques e exposições de dados. Em ambientes de

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

produção, é mais seguro usar um **PasswordEncoder** que aplica **hashing**, como **BCryptPasswordEncoder**, para garantir que as senhas armazenadas sejam mais difíceis de comprometer.

Agora, vamos configurar o provedor de autenticação **DaoAuthenticationProvider**?

```
@Bean
public DaoAuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
    authProvider.setUserDetailsService(myUserDetailService); // Define o UserDetailsService personalizado
    authProvider.setPasswordEncoder(passwordEncoder()); // Define o codificador de senha
    return authProvider;
}
```

Código-fonte 6 – DaoAuthenticationProvider
Fonte: elaborado pelo autor (2025)

@Bean

Marca o método authenticationProvider como um bean gerenciado pelo Spring. O DaoAuthenticationProvider será registrado no contêiner de IoC do Spring, disponível para ser injetado automaticamente onde necessário.

DaoAuthenticationProvider

É uma implementação específica do AuthenticationProvider no Spring Security. Ele usa o UserDetailsService para buscar detalhes do usuário e um PasswordEncoder para validar a senha do usuário.

authProvider.setUserDetailsService(myUserDetailService);

Define o UserDetailsService que o DaoAuthenticationProvider usará para carregar detalhes do usuário. Aqui, ele está configurado para usar o serviço personalizado myUserDetailService que criamos, permitindo que o Spring Security consulte as informações de autenticação.

authProvider.setPasswordEncoder(passwordEncoder());

Define o codificador de senha a ser usado. O PasswordEncoder é responsável por codificar a senha fornecida pelo usuário na autenticação e compará-la com a senha armazenada (normalmente, uma versão criptografada). Este método

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

pressupõe que você tenha um método passwordEncoder() em SecurityConfig que retorna um bean de PasswordEncoder, como BCryptPasswordEncoder.

return authProvider;

Retorna o DaoAuthenticationProvider totalmente configurado. Esse provedor será usado pelo AuthenticationManager para autenticar os usuários.

Resumindo, no método **authenticationProvider** estamos dizendo para o Spring Security que a autenticação será baseada na implementação da classe MyUserDetailService usando o usuário carregado pelo método loadUserByUsername dessa classe.

O método passwordEncoder define como a senha deve ser gerada e validada (hash, criptografia etc.). No exemplo, apenas para facilidade de entendimento, estamos usando a inner class **PlainTextPasswordEncoder**, indicando para o Spring Security que a senha estará em claro (com métodos para retornar e comparar strings puras). Ressaltamos que isso, no entanto, é uma má prática de desenvolvimento e uma aplicação real deve tomar cuidado para não trabalhar com senhas em aberto.

O Spring Security está configurado para validar se um usuário e senha são válidos antes de permitir acesso a nossa API. Agora, vamos para evolução da aplicação utilizando o Basic Authentication.

Vamos **modificar a implementação da classe de configuração SecurityConfig.** Como dito anteriormente, a partir do Spring Security 6.0, a classe **WebSecurityConfigurerAdapter** foi descontinuada. A configuração de segurança agora é feita usando a classe **SecurityFilterChain** e a configuração específica de autenticação é feita criando um **@Bean AuthenticationManager**.

O próximo passo é criar uma configuração de autenticação em memória usando a classe **InMemoryUserDetailsManager**, que é uma implementação de **UserDetailsManager** para autenticação de usuários em memória.

```
@Bean
public UserDetailsService userDetailsService() {
    //Configuração de usuário em memória
    InMemoryUserDetailsManager manager = new InMemoryUserDetailsManager();
    manager.createUser(User.withUsername("usuario")
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
            .password(passwordEncoder().encode("senha"))
            .roles("USER")
            .build());
    manager.createUser(User.withUsername("admin")
            .password(passwordEncoder().encode("password"))
            .roles("ADMIN", "USER")
            .build());
    return manager;
}
```

Código-fonte 7 – InMemoryUserDetailsManager
Fonte: elaborado pelo autor (2025)

@Bean

A anotação @Bean declara que o método userDetailsService deve ser gerenciado pelo contêiner de IoC (Inversion of Control) do Spring, ou seja, um objeto UserDetailsService será instanciado e gerenciado automaticamente pelo Spring quando a aplicação iniciar. Isso permite que o método seja chamado em outras partes do código onde uma instância de UserDetailsService é necessária.

public UserDetailsService userDetailsService()

Este é o método que retorna uma instância de UserDetailsService. Ele é responsável por criar e configurar o InMemoryUserDetailsManager, que gerenciará os usuários autenticados em memória.

InMemoryUserDetailsManager manager = new InMemoryUserDetailsManager();

Aqui, uma nova instância de InMemoryUserDetailsManager é criada. Esse gerenciador é utilizado para armazenar as credenciais de autenticação em memória, o que é útil para ambientes de teste ou para pequenas aplicações que não necessitam de um banco de dados de autenticação.

manager.createUser(...)

O método createUser é usado para adicionar novos usuários à instância manager. Cada chamada cria um usuário com um nome de usuário, senha e uma lista de papéis (roles).

return manager;

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Ao final, o método retorna o objeto manager, que contém os usuários configurados. Esse objeto é então disponibilizado como um UserDetailsService gerenciado pelo Spring e será usado pelo Spring Security para autenticação.

Observação: essa configuração em memória é prática para desenvolvimento e testes, mas em ambientes de produção é recomendável configurar o **UserDetailsService** para **buscar usuários em uma base de dados** ou em um **sistema de autenticação externo**.

Agora, precisamos modificar o método **authenticationManager.** Fizemos o comentário do código original:

```
/*@Bean
public AuthenticationManager
authenticationManager(AuthenticationConfiguration
authenticationConfiguration) throws Exception {
    return authenticationConfiguration.getAuthenticationManager();
}*/
```

Código-fonte 8 – Modificando o método authenticationManager
Fonte: elaborado pelo autor (2025)

Criando um novo código:

```
@Bean
public AuthenticationManager authenticationManager(HttpSecurity http) throws
Exception {
    AuthenticationManagerBuilder authenticationManagerBuilder =
            http.getSharedObject(AuthenticationManagerBuilder.class);

authenticationManagerBuilder.userDetailsService(userDetailsService()).passwordEncod
er(passwordEncoder());
    return authenticationManagerBuilder.build();
}
```

Código-fonte 9 – Modificando o método authenticationManager (2)
Fonte: elaborado pelo autor (2025)

Esse método define um **AuthenticationManager** configurado com um **UserDetailsService** e um **PasswordEncoder**. Vamos analisar cada linha em detalhes.

@Bean

A anotação @Bean indica que o método authenticationManager deve ser gerenciado pelo contêiner de IoC do Spring. Isso significa que uma instância de

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

AuthenticationManager será criada automaticamente e estará disponível para injeção em outras partes da aplicação.

public AuthenticationManager authenticationManager(HttpSecurity http) throws Exception

Esse é o método que define e retorna um AuthenticationManager. Ele recebe um parâmetro HttpSecurity, que é a principal classe para configurar a segurança HTTP no Spring Security. O método pode lançar uma exceção (throws Exception) caso ocorra algum problema durante a configuração.

AuthenticationManagerBuilder authenticationManagerBuilder = http.getSharedObject(AuthenticationManagerBuilder.class);

Aqui, o método getSharedObject é chamado no objeto http para obter uma instância de AuthenticationManagerBuilder. O AuthenticationManagerBuilder é utilizado para construir e configurar o AuthenticationManager, que é responsável por gerenciar o processo de autenticação. Esse compartilhamento do AuthenticationManagerBuilder permite configurar autenticações de forma centralizada para que o HttpSecurity e outros componentes do Spring Security possam utilizá-lo.

authenticationManagerBuilder.userDetailsService(userDetailsService()).passwordEncoder(passwordEncoder());

Esse comando configura o AuthenticationManagerBuilder para utilizar o UserDetailsService e o PasswordEncoder da aplicação.

userDetailsService(userDetailsService())

Configura o AuthenticationManagerBuilder com um UserDetailsService personalizado (neste caso, o método userDetailsService() definido em outra parte da aplicação). O UserDetailsService fornece os detalhes do usuário (nome de usuário, senha, roles etc.) para o processo de autenticação.

passwordEncoder(passwordEncoder())

Configura o AuthenticationManagerBuilder com um PasswordEncoder, que é responsável por codificar e verificar as senhas. A utilização de um PasswordEncoder

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

garante que as senhas não sejam armazenadas em texto plano, aumentando a segurança.

return authenticationManagerBuilder.build();

O método build() é chamado no authenticationManagerBuilder para construir e retornar uma instância de AuthenticationManager completamente configurada. Esse **AuthenticationManager** resultante será gerenciado pelo Spring e utilizado para autenticação dos usuários. Ele será responsável por gerenciar as autenticações em toda a aplicação, sendo usado pelo Spring Security para verificar as credenciais dos usuários.

Precisamos comentar o método **authenticationProvider().**

```
/*@Bean
public DaoAuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider authProvider = new
DaoAuthenticationProvider();
    authProvider.setUserDetailsService(myUserDetailService); // Define o
UserDetailsService personalizado
    authProvider.setPasswordEncoder(passwordEncoder()); // Define o
codificador de senha
    return authProvider;
}*/
```

Código-fonte 10 – authenticationProvider()
Fonte: elaborado pelo autor (2025)

Por fim, precisamos adicionar a linha abaixo no método **securityFilterChain().**

```
http.httpBasic(Customizer.withDefaults());
```

Também precisamos comentar o **http.formLogin(padrão):**

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
    http.authorizeHttpRequests(
            customizer -> {
                customizer.requestMatchers("/public").permitAll();
                customizer.requestMatchers("/logout").permitAll();
                customizer.anyRequest().authenticated();
            });
    http.httpBasic(Customizer.withDefaults());
    //http.formLogin(Customizer.withDefaults());
    //http.cors(Customizer.withDefaults());
    //http.csrf(AbstractHttpConfigurer::disable);
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
    return http.build();
}
```

Código-fonte 11 – http.formLogin(padrão)
Fonte: elaborado pelo autor (2025)

Por fim, criamos um serviço com autenticação via formulário e controle de acesso por perfis de usuário.

Iniciamos com a criação das entidades **User** e **Role** para gerenciar **usuários** e suas **permissões**. Ao abrir o projeto, nos deparamos com a classe Main Application **RbacApplication** criada. Essa é a classe principal para iniciar o projeto.

A seguir, vamos criar a classe de entidade **Role**, que é uma representação do perfil**.** Iniciamos com a criação do pacote selecionando o principal **com.fiap.rbac** e as opções **New → Package**. Digitamos o nome do pacote **com.fiap.rbac.entity** e selecionamos o pacote criado e as opções **New → Java Class**. Então selecionamos a opção **Class** e digitamos o nome da classe **Role**. Não esqueça de gerar os Getters e Setters para as classes USER e ROLES.

```
package com.fiap.rbac.entity;

import jakarta.persistence.*;

@Entity

@Table(name = "tb_role")
public class Role {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true)
    private String name;

    // Getters e Setters
}
```

Código-fonte 12 – Criar a classe de entidade Role
Fonte: elaborado pelo autor (2025)

A seguir, vamos criar a classe de entidade **User**, que é uma representação do usuário**.** Iniciamos com a seleção do pacote **com.fiap.rbac.entity** e as opções **New → Java Class**, selecione a opção **Class** e digite o nome da classe **User**.

```
package com.fiap.rbac.entity;

import jakarta.persistence.*;
import java.util.Set;

@Entity
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```java
@Table(name = "tb_user")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String username;
    private String password;
    private boolean enabled;




    @ManyToMany(cascade = CascadeType.ALL, fetch = FetchType.EAGER)
    @JoinTable(name = "user_roles",
            joinColumns = @JoinColumn(name = "user_id"),
            inverseJoinColumns = @JoinColumn(name = "role_id"))
    private Set<Role> roles;

    // Getters e Setters
}
```

Código-fonte 13 – Criando a classe de entidade **User**
Fonte: elaborado pelo autor (2025)

Cabe observar a anotação **@ManyToMany** usada para configurar o relacionamento do usuário com as permissões. Com ela, um usuário poderá ter várias permissões e várias permissões estarão associadas a vários usuários. Teremos no banco dados uma tabela associativa intermediária entre a tabela de usuários e roles, contendo a propriedade **name** com o nome dessa tabela: **user_roles**. Em joinColumns está indicado qual é a coluna em que vai entrar a chave do usuário **user_id** e, para compor a indicação da outra coluna para fazer parte da chave primária, temos o **inverseJoinColumns**, contendo o id da permissão que é **role_id**.

Além disso, é preciso definir o **cascade**, ou seja, indicar que todas as operações de persistência (salvar, atualizar, deletar) realizadas em **User** também serão aplicadas nas entidades **Role** associadas. Podemos também definir a forma como vamos trazer os dados do banco com o **EAGER**, especificando que, ao buscar um usuário, as informações de **roles** serão carregadas imediatamente.

A seguir, vamos criar os repositórios? Iniciamos com a criação do pacote selecionando o principal **com.fiap.rbac** e as opções **New → Package**. Digitamos o nome do pacote **com.fiap.rbac.repository**, selecionamos o pacote criado e as opções **New → Java Class**. Então selecionamos a opção **Class** e digitamos o nome da classe **UserRepository**.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```java
package com.fiap.rbac.repository;

import com.fiap.rbac.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
}
```

Código-fonte 14 – Criando repositórios
Fonte: elaborado pelo autor (2025)

A seguir, selecione o pacote **com.fiap.rbac.repository** e as opções **New → Java Class**. Selecione então a opção **Class** e digite o nome da classe **RoleRepository**.

```java
package com.fiap.rbac.repository;

import com.fiap.rbac.entity.Role;
import org.springframework.data.jpa.repository.JpaRepository;

public interface RoleRepository extends JpaRepository<Role, Long> {}
```

Código-fonte 15 – Criando repositórios (2)
Fonte: elaborado pelo autor (2025)

O próximo passo é configurar o serviço de usuário e a autenticação. Para isso, vamos implementar a **UserDetailsService**, que é uma interface que faz parte da biblioteca do Spring Security e define o método **loadUserByUsername(String username)** para buscar detalhes do usuário pelo nome de usuário. Assim, vamos personalizar um serviço para carregar os usuários com base em sua **Role**.

Iniciamos com a criação do pacote selecionando o principal **com.fiap.rbac** e as opções **New → Package**. Digitamos o nome do pacote **com.fiap.rbac.service.** Selecionamos o pacote criado e as opções **New → Java Class**. Selecionamos a opção **Class** e digitamos o nome da classe **CustomUserDetailsService**.

```java
package com.fiap.rbac.service;
import com.fiap.rbac.entity.User;
import com.fiap.rbac.repository.UserRepository;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.stereotype.Service;
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
import java.util.stream.Collectors;

@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    public CustomUserDetailsService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
        // Busca o usuário pelo nome de usuário
        User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new UsernameNotFoundException("User not found
with username: " + username));

        // Converte as roles do usuário para SimpleGrantedAuthority
        var authorities = user.getRoles().stream()
                .map(role -> new SimpleGrantedAuthority(role.getName()))
                .collect(Collectors.toList());

        // Retorna uma implementação do UserDetails do Spring Security com os dados
do usuário
        return new org.springframework.security.core.userdetails.User(
                user.getUsername(),
                user.getPassword(),
                user.isEnabled(),      // habilitado
                true,                  // conta não expirada
                true,                  // credenciais não expiradas
                true,                  // conta não bloqueada
                authorities
        );
    }
}
```

Código-fonte 16 – Implementando UserDetailsService
Fonte: elaborado pelo autor (2025)

Injeção de dependência: CustomUserDetailsService recebe o UserRepository através do construtor, permitindo que o serviço carregue dados do usuário do banco de dados.

Método loadUserByUsername: esse método busca o usuário no banco pelo nome de usuário fornecido. Se o usuário não for encontrado, uma exceção UsernameNotFoundException é lançada.

Mapeamento das Roles para Authorities: as roles do usuário são convertidas em uma lista de SimpleGrantedAuthority. Cada role é mapeada para um SimpleGrantedAuthority com o nome da role (exemplo: ROLE_ADMIN). Isso é necessário porque o Spring Security usa GrantedAuthority para verificar as permissões de acesso.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Criação do UserDetails: retorna uma instância de User do Spring Security (não confundir com nossa entidade User). Essa implementação de User inclui:

- **username**: o nome de usuário do banco de dados.
- **password**: a senha criptografada do usuário, se necessário.
- **enabled**: indica se o usuário está ativo.
- **accountNonExpired**, **credentialsNonExpired**, **accountNonLocked**: configurados como true para simplificação, mas podem ser ajustados conforme o comportamento desejado.
- **authorities**: lista de roles mapeadas.

O próximo passo é configurar a segurança com o Spring Security. Para isso, vamos criar uma classe de configuração para configurar o Spring Security e definir as permissões de acesso.

Iniciamos com a criação do pacote selecionando o principal **com.fiap.rbac** e as opções **New → Package**. Digitamos o nome do pacote **com.fiap.rbac.config.** Selecionamos o pacote criado e as opções **New → Java Class**, depois a opção **Class** e digitamos o nome da classe **SecurityConfig**.

Essa é a classe de configuração do Spring, então temos que adicionar a anotação **@Configuration**. Ela terá outra anotação especial, que é **@EnableWebSecurity**, onde será reconhecida como a classe que habilita a segurança para o serviço web. Antes era preciso estender a classe **WebSecurityConfigurerAdapter**, mas ela foi removida na nova versão do Spring Security.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig { }
```

Código-fonte 17 – SecurityConfig
Fonte: elaborado pelo autor (2025)

Além disso, temos o código a seguir dentro da classe:

```
private final CustomUserDetailsService userDetailsService;

public SecurityConfig(CustomUserDetailsService userDetailsService) {
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
    this.userDetailsService = userDetailsService;

}
```

Código-fonte 17 – userDetailsService
Fonte: elaborado pelo autor (2025)

private final CustomUserDetailsService userDetailsService;

Esta linha declara uma variável userDetailsService do tipo CustomUserDetailsService, que será usada para fornecer os detalhes do usuário durante a autenticação. O uso do modificador final indica que essa variável é imutável após sua atribuição.

public SecurityConfig(CustomUserDetailsService userDetailsService)

Este é o construtor da classe SecurityConfig. Ele injeta a dependência CustomUserDetailsService, que é um serviço que implementa a interface UserDetailsService e carrega os detalhes do usuário (como nome e senha). Esse serviço é essencial para o Spring Security gerenciar a autenticação baseada em usuários.

O próximo passo é criar o filtro, semelhante ao que já fizemos no projeto anterior.

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception
{
    http
            .authorizeHttpRequests(authorizeRequests ->
                    authorizeRequests
                            .requestMatchers("/h2-console/**").permitAll() //
Permitir acesso ao console H2
                            .requestMatchers("/admin/**").hasRole("ADMIN")
                            .requestMatchers("/user/**").hasRole("USER")
                            .anyRequest().authenticated()
            )
            .formLogin(form -> form
                    .successHandler(customAuthenticationSuccessHandler()) // Define
o redirecionamento pós-login
            )
            .logout(logout -> logout.permitAll());


    // Desativa a proteção de frameOptions para permitir que o console H2 funcione
corretamente,
    // já que ele utiliza frames para exibir o conteúdo.
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
    http.headers(headers -> headers.frameOptions().disable());
    // Desabilitar CSRF no console H2, necessário para que o H2 funcione
corretamente
    http.csrf(csrf -> csrf.ignoringRequestMatchers("/h2-console/**"));

    return http.build();
}
```

Código-fonte 19 – Criando filtro
Fonte: elaborado pelo autor (2025)

Essa parte é importante, pois a segurança é feita através de filtros (**filters**). Nos **filters** entram requisições web, que as interceptam e checam se elas são autenticadas ou não. Se não forem, o controlador nem executa a lógica de negócio na service.

Para isso, vamos adicionar um **@Bean** chamado **SecurityFilterChain.** Novamente, é uma chain porque são vários filtros de segurança, uma cadeia para customizar a autenticação e autorização. Ele recebe como parâmetro um objeto **HttpSecurity** usado para configurar a segurança. Vou usar a sintaxe de **Lambda Function** para expor a configuração de autorização para cada rota. Ao final faço um **Build** para retornar o resultado.

Agora, vamos implementar o método que permite a configuração de um **AuthenticationSuccessHandler** para redirecionar usuários com base na role.

```
// Configuração de um AuthenticationSuccessHandler para redirecionar usuários com
base na role
@Bean
public AuthenticationSuccessHandler customAuthenticationSuccessHandler() {
    // Retorna uma nova instância de AuthenticationSuccessHandler anônimo
    return new AuthenticationSuccessHandler() {
        // Método que será executado quando a autenticação for bem-sucedida
        @Override
        public void onAuthenticationSuccess(HttpServletRequest request,
HttpServletResponse response,
                                            Authentication authentication) throws
IOException, ServletException {
            // Verifica se o usuário autenticado possui a role "ROLE_ADMIN"
            if (authentication.getAuthorities().contains(new
SimpleGrantedAuthority("ROLE_ADMIN"))) {
                // Redireciona o usuário com a role "ROLE_ADMIN" para a página
"/admin"
                response.sendRedirect("/admin");
                // Verifica se o usuário possui a role "ROLE_USER"
            } else if (authentication.getAuthorities().contains(new
SimpleGrantedAuthority("ROLE_USER"))) {
                // Redireciona o usuário com a role "ROLE_USER" para a página
"/user"
                response.sendRedirect("/user");
                // Caso o usuário não tenha uma role específica definida
            } else {
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
                // Redireciona o usuário para a raiz "/" como fallback padrão
                response.sendRedirect("/");
            }
        }
    };
}
```

Código-fonte 20 – AuthenticationSuccessHandler
Fonte: elaborado pelo autor (2025)

Criação de um Bean: este método está marcado com a anotação **@Bean**, o que significa que o **customAuthenticationSuccessHandler** será gerenciado pelo Spring, permitindo injeção em outros componentes, se necessário.

Implementação de um AuthenticationSuccessHandler: o **AuthenticationSuccessHandler** é uma interface do Spring Security usada para definir ações após uma autenticação bem-sucedida. Neste caso, estamos criando uma implementação anônima diretamente dentro do método, que permite customizar o comportamento do redirecionamento.

Método onAuthenticationSuccess: esse método é invocado automaticamente quando a autenticação é bem-sucedida. Ele recebe três parâmetros principais:

- **HttpServletRequest request**: contém os dados da requisição HTTP.
- **HttpServletResponse response**: permite modificar a resposta, como fazer redirecionamentos.
- **Authentication authentication**: contém os detalhes da autenticação do usuário, incluindo informações sobre as permissões ou roles.

Verificação de Roles (Autorização): usando **authentication.getAuthorities().contains(newSimpleGrantedAuthority("ROLE_ADMIN"))**, verificamos se o usuário possui a role "**ROLE_ADMIN**". Se a role estiver presente, o usuário será redirecionado para a URL "**/admin**". Se a role "**ROLE_USER**" for encontrada, o usuário será redirecionado para a URL "**/user**".

Redirecionamento para a raiz como fallback: caso o usuário não possua uma role específica configurada (ou se houver roles diferentes das tratadas), o redirecionamento padrão levará o usuário à página raiz "**/**".

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Agora, precisamos de métodos que definem uma implementação personalizada de um **PasswordEncoder**, que armazena senhas em texto puro, sem qualquer tipo de criptografia.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new PlainTextPasswordEncoder();
}

public static class PlainTextPasswordEncoder implements PasswordEncoder {
    @Override
    public String encode(CharSequence rawPassword) {
        return rawPassword.toString(); // Retorna a senha como está, sem criptografia
    }

    @Override
    public boolean matches(CharSequence rawPassword, String encodedPassword) {
        return rawPassword.toString().equals(encodedPassword); // Compara diretamente a senha
    }
}
```

Código-fonte 21 – PasswordEncoder
Fonte: elaborado pelo autor (2025)

passwordEncoder

Este método está anotado com @Bean, o que indica ao Spring que ele deve ser gerenciado como um bean no contexto da aplicação. Ele retorna uma instância da **inner class PlainTextPasswordEncoder**, que implementa a interface **PasswordEncoder** do Spring Security. **PasswordEncoder** é uma interface que define métodos para codificação e verificação de senhas, geralmente usada para aplicar hashing ou criptografia nas senhas antes de armazená-las.

PlainTextPasswordEncoder

Esta é uma classe estática interna que implementa a interface **PasswordEncoder**. Como vimos, ela não realiza nenhuma criptografia ou hashing, armazenando as senhas em texto puro.

Método encode

Recebe a senha bruta (rawPassword) como um CharSequence e a converte para uma String usando toString(). Em seguida, retorna o valor da senha exatamente como foi recebida, sem aplicar criptografia ou hashing.

Método matches

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Recebe dois parâmetros: **rawPassword**, que é a senha fornecida pelo usuário, e **encodedPassword**, que é a senha armazenada (não criptografada, neste caso). O método compara diretamente os valores da senha fornecida com a senha armazenada usando equals(). Isso significa que as senhas precisam coincidir exatamente em termos de texto. Como não há hashing ou criptografia, o método matches apenas verifica se as duas strings são idênticas.

Agora, vamos configurar o provedor de autenticação **DaoAuthenticationProvider**.

```
@Bean
public DaoAuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
    authProvider.setUserDetailsService(userDetailsService); // Define o UserDetailsService personalizado
    authProvider.setPasswordEncoder(passwordEncoder()); // Define o codificador de senha sem criptografia
    return authProvider;
}
```

Código-fonte 22 – DaoAuthenticationProvider
Fonte: elaborado pelo autor (2025)

@Bean

Marca o método **authenticationProvider** como um bean gerenciado pelo Spring. O **DaoAuthenticationProvider** será registrado no contêiner de IoC do Spring, disponível para ser injetado automaticamente onde necessário.

DaoAuthenticationProvider

Implementação específica do AuthenticationProvider no Spring Security, usa o UserDetailsService para buscar detalhes do usuário e um PasswordEncoder para validar a senha do usuário.

authProvider.setUserDetailsService(userDetailsService);

Define o UserDetailsService que o DaoAuthenticationProvider usará para carregar detalhes do usuário. Aqui, ele está configurado para usar o serviço personalizado myUserDetailService que criamos, permitindo que o Spring Security consulte as informações de autenticação.

authProvider.setPasswordEncoder(passwordEncoder());

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Define o codificador de senha a ser usado. O PasswordEncoder é responsável por codificar a senha fornecida pelo usuário na autenticação e compará-la com a senha armazenada (normalmente, uma versão criptografada). Este método pressupõe que você tenha um método passwordEncoder() em SecurityConfig que retorna um bean de PasswordEncoder, como BCryptPasswordEncoder.

return authProvider;

Retorna o DaoAuthenticationProvider totalmente configurado. Esse provedor será usado pelo AuthenticationManager para autenticar os usuários.

Resumindo, no método authenticationProvider estamos dizendo para o Spring Security que a autenticação será baseada na implementação da classe CustomUserDetailsService, usando o usuário carregado pelo método loadUserByUsername dessa classe.

Assim como vimos anteriormente, o método passwordEncoder define como a senha deve ser gerada e validada. Apenas para facilitar entendimento, estamos usando a inner class **PlainTextPasswordEncoder** que indica para o Spring Security que a senha estará em claro, o que é uma má prática de desenvolvimento e não deve ser feito em uma aplicação real.

O próximo passo é configurar o banco de dados e realizar a carga inicial de dados. Para isso, crie um arquivo data.sql em src/main/resources para inserir usuários e roles de exemplo.

```
INSERT INTO tb_role (id, name) VALUES (1, 'ROLE_ADMIN');
INSERT INTO tb_role (id, name) VALUES (2, 'ROLE_USER');

INSERT INTO tb_user (id, username, password, enabled) VALUES (1, 'admin', '123', true); --
Senha SEM criptografia
INSERT INTO user_roles (user_id, role_id) VALUES (1, 1);

INSERT INTO tb_user (id, username, password, enabled) VALUES (2, 'user', '456', true); --
Senha SEM criptografia
INSERT INTO user_roles (user_id, role_id) VALUES (2, 2);
```

Código-fonte 23 – Configurar o banco de dados
Fonte: elaborado pelo autor (2025)

Agora criamos o controlador (**Controller**) e configuramos templates HTML. Para isso, crie um controlador para redirecionar strings das páginas de **admin**, **user** e **login**.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```java
package com.fiap.rbac.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HomeController {

    @GetMapping("/admin")
    public String adminPage() {
        return "<h1>ADMIN, você está acessando a área administrativa do app!</h1>";
    }



    @GetMapping("/user")
    public String userPage() {
        return "<h1>USER, você está acessando a área geral do app!</h1>";
    }

    @GetMapping("/login")
    public String loginPage() {
        return "login";
    }
}
```

Código-fonte 24 – Controlador
Fonte: elaborado pelo autor (2025)

Ao final, vamos editar o arquivo **Application.properties**.

```properties
spring.application.name=rbac

# Permitir a execução dos comandos no data.sql
spring.sql.init.mode=always

# Configurações do Banco de Dados H2 (apenas para testes)
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# JPA (Java Persistence API) Configurações
spring.jpa.hibernate.ddl-auto=update
# Atrasa a execução do script no data.sql para fazer após a criação das tabelas
spring.jpa.defer-datasource-initialization=true
spring.jpa.show-sql=true

# Configurações do Spring Security
# Define a URL de login personalizada (caso precise ser usada em outra parte do sistema)
spring.security.form-login.page=/login

# Configurações de logging (opcional)
logging.level.org.springframework.security=DEBUG

# Configuração de mensagem para login personalizado (opcional)
spring.messages.basename=messages
```

Código-fonte 25 – Application.properties
Fonte: elaborado pelo autor (2025)

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Discutimos os principais desafios na Segurança em Aplicações Java com Spring Security na versão atualizada (6.1). Navegamos pelas estruturas (classes/interfaces) do Spring Security, aprendemos a preparar o ambiente de desenvolvimento e vimos todos os códigos necessários para codificação dos projetos. Além disso, utilizamos os principais comandos para manutenção do banco de dados, envolvendo as entidades **User** e **Roles** no projeto RBAC.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

SPRING.IO **Documentação do Spring Security**. Disponível em: https://spring.io/projects/spring-security. Acesso em: 25 mar. 2025.

NARDONE, M; SCARIONI, C. **Pro Spring Security: Securing Spring Framework 6 and Boot 3-based Java Applications**. New York: Apress, 2023.

BAELDUNG. **Guia do Baeldung**. Disponível em: https://www.baeldung.com/?s=spring%20security. Acesso em: 25 mar. 2025.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Palavras-chave: spring. security. authentication. rbac.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com